

Product Requirements Document (PRD)

Project Title

Next-Gen AI Platform Landing Page Speed Run

Document Version

1.0

Date

26 June 2026

Objective

Build a premium, high-converting, fully responsive SaaS landing page for an advanced AI-driven data automation platform. The landing page must demonstrate strong engineering architecture, dynamic pricing logic, responsive UI transformation, motion polish, SEO hygiene, and strict performance discipline.

The final product should feel like a modern enterprise AI platform website while meeting all functional, technical, asset, and performance constraints defined for the round.

1. Product Overview

1.1 Product Concept

The product is an **AI-driven data automation platform** designed for businesses that want to automate workflows, process data intelligently, and improve operational performance using next-generation AI systems.

The website must present the platform as premium, reliable, technically advanced, and conversion-focused.

1.2 Target Users

- Startup founders and operators
- Enterprise decision-makers

- Data teams and automation engineers
- SaaS buyers comparing AI automation tools
- Technical evaluators reviewing architecture and performance

1.3 Primary User Goal

Users should quickly understand:

- What the AI platform does
- Why it is valuable
- Which features are available
- How pricing changes by currency and billing cycle
- Whether the product is trustworthy and production-ready

1.4 Business Goal

Create a high-quality landing page that maximizes credibility, conversion, and technical evaluation score.

2. Scope

2.1 In Scope

The landing page must include:

- Hero section
- Technical feature showcase
- Bento-grid feature layout on desktop
- Accordion feature layout on mobile
- Dynamic pricing tier matrix
- Monthly/annual billing toggle
- INR, USD, and EUR currency switcher
- Social proof section
- Responsive design across mobile, tablet, and desktop
- SEO metadata and semantic HTML
- Native CSS or WAAP-based animations
- Asset usage from the provided asset package

- Public GitHub repository
- Live deployment link
- Demo video link under 100MB

2.2 Out of Scope

The following are not required:

- Full backend system
 - User authentication
 - Real payment integration
 - Real dashboard application
 - CMS integration
 - External database
 - Production analytics integration
-

3. Functional Requirements

3.1 Landing Page Structure

Requirement

The page must be a cohesive SaaS landing page with strong visual storytelling.

Mandatory Sections

1. Header / Navigation

- Brand identity
- Navigation links
- Primary call-to-action
- Responsive mobile navigation if required

2. Hero Section

- Clear headline communicating AI automation value
- Supporting subheadline
- Primary and secondary CTA buttons
- Premium visual element using provided SVG/UI assets

- Fast entry animation within performance limits

3. Feature Showcase

- Technical product features presented visually
- Desktop: Bento-grid layout
- Mobile: Accordion layout
- State persistence across breakpoint transition

4. Pricing Section

- Pricing cards or tier matrix
- Billing toggle: Monthly / Annual
- Currency selector: INR, USD, EUR
- Dynamic price calculation from configuration matrix
- Annual pricing must apply a flat 20% discount

5. Social Proof Section

- Testimonials, usage metrics, logos, ratings, or trust statements
- Must use crawlable text where possible

6. Footer

- Product links
- Company links
- Legal or copyright text
- Semantic footer markup

3.2 Feature 1: Matrix-Driven Pricing & Performance-Isolated Currency Switcher

Requirement

Implement a dynamic pricing system that supports multiple tiers, billing cycles, and currencies.

Pricing Inputs

The pricing engine must use a multi-dimensional configuration object or matrix containing:

- Base tier rate
- Currency symbol
- Regional tariff variables
- Billing cycle multiplier
- Annual discount multiplier

Supported Billing Cycles

- Monthly
- Annual

Supported Currencies

- INR: ₹
- USD: \$
- EUR: €

Annual Discount Rule

Annual billing must apply a flat **20% discount**.

Recommended formula:

```
finalPrice = baseTierRate * regionalTariff * billingCycleMultiplier *  
annualDiscountMultiplier
```

Where:

- Monthly discount multiplier = 1
- Annual discount multiplier = 0.8

Critical Performance Constraint

Changing the billing cycle or currency must **not trigger global re-renders**.

Required Behavior

- Price text nodes should update locally only.
- Parent layout, hero, feature section, and surrounding sections must not re-render or reflow.
- Pricing data must not be hardcoded directly into UI markup.

- Currency and billing changes must be derived from the matrix.

Acceptance Criteria

- User can switch between Monthly and Annual.
 - User can switch between INR, USD, and EUR.
 - All displayed prices update correctly.
 - Annual prices reflect 20% discount.
 - Chrome DevTools should show updates isolated to price text nodes.
 - No global component remounting or layout flash occurs.
-

3.3 Feature 2: Bento-to-Accordion Wrapper with State Persistence

Requirement

Present product features in a Bento-grid on desktop and automatically refactor into an accordion on mobile.

Desktop Behavior

- Features appear as a modern Bento-grid.
- Hovering or focusing a Bento item sets it as the active item.
- Active item may show expanded detail, visual emphasis, or motion feedback.

Mobile Behavior

- Features appear as a touch-friendly accordion list.
- Only relevant panels should expand based on active state.
- Accordion interaction must be smooth and accessible.

Context Lock Constraint

If a user is hovering over or interacting with a specific Bento item on desktop and then resizes the browser into the mobile breakpoint, the application must transfer that exact active index to the accordion.

Example

1. User hovers Feature Card 3 on desktop.

2. User resizes browser below the mobile breakpoint.
3. Mobile accordion appears.
4. Accordion panel 3 is automatically open.

Dependency Constraint

This feature must be built from scratch.

Not Allowed

- Shadcn
- Radix UI
- HeadlessUI
- Framer Motion
- Tailwind UI prebuilt components
- Any external UI component library for the core Bento/Accordion component
- Any runtime animation engine for the core motion behavior

Acceptance Criteria

- Desktop uses Bento-grid layout.
 - Mobile uses accordion layout.
 - Active index persists during breakpoint transition.
 - Component is accessible by keyboard and pointer.
 - No banned UI or animation libraries are used.
 - Transitions are implemented using native CSS transitions/animations or WAAPL.
-

4. UX Requirements

4.1 Visual Style

The interface should communicate:

- Premium SaaS quality
- AI-native intelligence
- Enterprise trust
- Technical sophistication
- High conversion intent

4.2 Layout Requirements

- Fully responsive across mobile, tablet, and desktop
- No horizontal overflow
- No overlapping typography
- Fluid breakpoint adaptation
- Strong visual hierarchy
- Clear CTAs

4.3 Motion Requirements

Motion should enhance clarity without harming performance.

Motion Type	Duration	Easing
Hover interactions	150ms – 200ms	ease-out
Toggle interactions	150ms – 200ms	ease-out
Structural layout reflows	300ms – 400ms	ease-in-out
Initial loading/entry sequence	Max 500ms total	Performance-safe

4.4 Accessibility Requirements

- Use semantic landmarks: header , main , section , article , nav , footer
 - Interactive controls must be keyboard accessible
 - Buttons and toggles must have accessible names
 - Accordion must expose expanded/collapsed state
 - Images/SVGs must have meaningful alt text or be hidden when decorative
 - Text contrast must be readable against the provided color palette
-

5. SEO Requirements

5.1 Metadata

The page must include:

- Descriptive page title

- Meta description
- Viewport tag
- Open Graph title
- Open Graph description
- Open Graph image if available from assets
- Twitter card metadata if applicable

5.2 Semantic HTML

The DOM must prioritize meaningful structure over excessive nested `div` usage.

Required semantic tags:

- `header`
- `nav`
- `main`
- `section`
- `article`
- `footer`

5.3 Crawlability

- Core marketing copy must be rendered as crawlable text.
- Pricing labels and plan names should be readable in the DOM.
- Important content should not be hidden behind canvas-only visuals.

6. Technical Requirements

6.1 Allowed Frameworks

Any one of the following may be used:

- React
- Next.js
- Vue
- Nuxt
- SolidJS

- Angular
- Vanilla JavaScript

6.2 Styling Options

Allowed:

- Tailwind CSS
- Bootstrap
- Custom CSS
- CSS variables

6.3 Animation Options

Allowed:

- Native CSS transitions
- Native CSS animations
- Web Animations API

Not allowed:

- Runtime CSS-in-JS animation engines
- Framer Motion
- Prebuilt animation component libraries

6.4 3D/Visual Enhancements

Allowed:

- Three.js or similar 3D frameworks, if used responsibly

6.5 Performance Constraints

- Initial loading and entry animation must complete within 500ms.
 - Animations must not block semantic HTML indexing.
 - Time to Interactive must not be delayed by motion choreography.
 - Avoid layout thrashing.
 - Avoid excessive component mount/unmount flashing.
 - Avoid global re-renders from pricing controls.
-

7. Asset Requirements

The implementation must meaningfully use all provided asset categories.

Required Asset Categories

1. SVG Pack

- Use provided SVGs or UI elements in the interface.

2. Font List

- Configure the specified two primary font families through CSS or document head layers.

3. Color Palette

- Apply the predefined hex color palette consistently throughout the UI.

4. Demo Video Reference

- Match the structural layout motion and transitions as closely as possible.

Asset Compliance Acceptance Criteria

- Assets are not merely imported; they are visibly and meaningfully integrated.
 - Typography follows the provided font requirements.
 - Colors follow the provided palette.
 - No external resources are used unless explicitly permitted by the challenge.
-

8. Data Model Requirements

8.1 Pricing Configuration Example

```
const pricingMatrix = {
  currencies: {
    INR: { symbol: '₹', tariff: 1 },
    USD: { symbol: '$', tariff: 0.012 },
    EUR: { symbol: '€', tariff: 0.011 }
  },
  billing: {
    monthly: { multiplier: 1, discount: 1 },
    annual: { multiplier: 12, discount: 0.8 }
  },
  tiers: {
    starter: { baseRate: 2999 },
    growth: { baseRate: 7999 },
    enterprise: { baseRate: 19999 }
  }
};
```

8.2 Pricing Calculation Example

```
function calculatePrice(tier, currency, billingCycle) {
  const base = pricingMatrix.tiers[tier].baseRate;
  const tariff = pricingMatrix.currencies[currency].tariff;
  const cycle = pricingMatrix.billing[billingCycle];

  return base * tariff * cycle.multiplier * cycle.discount;
}
```

8.3 State Isolation Requirement

Pricing control state should be scoped to the pricing component or managed with direct DOM text-node updates, memoization, refs, signals, or other localized update mechanisms depending on framework choice.

9. Non-Functional Requirements

9.1 Performance

- No unnecessary global re-renders
- Minimal JavaScript for critical UI
- CSS-first animations
- Hardware-accelerated transforms where possible
- Avoid animation of expensive layout properties when possible

9.2 Maintainability

- Clear component structure
- Reusable pricing configuration
- Documented calculation logic
- Meaningful naming conventions
- Separation of layout, logic, and data

9.3 Reliability

- Site must build successfully
- Live deployment must load without 404 or 500 errors
- Repository must be public and auditable

9.4 Responsiveness

Required viewport support:

- Mobile: 320px and above
 - Tablet: 768px and above
 - Desktop: 1024px and above
 - Large desktop: 1440px and above
-

10. Suggested Page Content Direction

10.1 Hero Copy Example

Headline: Automate Data Workflows with Next-Generation AI

Subheadline: Connect, transform, and orchestrate business data using intelligent automation built for modern teams.

10.2 Feature Themes

Possible feature cards:

- AI workflow orchestration
- Real-time data pipelines
- Predictive automation engine
- Enterprise-grade security
- No-code integration layer
- Observability and performance insights

10.3 Pricing Tiers

Suggested tiers:

- Starter
- Growth
- Enterprise

Each tier should include:

- Plan name
 - Dynamic price
 - Billing period
 - Feature list
 - CTA button
-

11. Acceptance Criteria

11.1 Functional Acceptance

- Landing page contains all required sections.
- Pricing engine uses a matrix-based configuration.
- Currency switcher supports INR, USD, and EUR.
- Billing toggle supports monthly and annual billing.
- Annual billing applies 20% discount.
- Bento layout appears on desktop.
- Accordion layout appears on mobile.
- Active Bento index transfers correctly to mobile accordion.

11.2 Technical Acceptance

- No banned UI or animation libraries are used for core components.
- Pricing state updates are isolated.
- No global layout reflow occurs on pricing changes.
- Initial animation sequence completes within 500ms.
- Native CSS transitions/animations or WAAP API are used.
- Semantic HTML structure is present.
- SEO metadata is implemented.

11.3 Design Acceptance

- Page feels premium and cohesive.
- Provided assets are meaningfully used.
- Typography follows provided font list.
- Color palette follows provided hex codes.
- Layout is fluid and responsive.
- Motion timing follows specified constraints.

12. Scoring Alignment

Category	Points	PRD Coverage
Logic, Architecture & State Isolation	40	Pricing matrix, isolated updates, Bento-to-Accordion state persistence
SEO Optimization & Semantic HTML	30	Metadata, semantic landmarks, crawlable content
UI/UX Usability & Motion Matching	30	Responsive layout, asset compliance, native motion, breakpoint fluidity

13. Risks & Mitigation

Risk: Pricing controls trigger parent re-render

Mitigation: Isolate pricing state, use memoization/refs/signals/direct text-node updates, avoid lifting pricing state globally.

Risk: Accordion state does not persist after resize

Mitigation: Maintain a shared active index independent of layout mode and update it on hover/focus/click.

Risk: Banned dependencies accidentally included

Mitigation: Audit `package.json` before submission.

Risk: Animations harm TTI

Mitigation: Use CSS transforms and opacity, keep entry animation under 500ms, avoid heavy JavaScript on load.

Risk: Asset compliance score loss

Mitigation: Map each provided asset category to a visible page section before implementation.

14. Deliverables

Final submission must include:

1. Public GitHub repository link
 2. Working live deployment link
 3. Demo video link under 100MB
 4. Complete source code
 5. Integrated provided assets
 6. Responsive landing page implementation
-

15. Disqualification Constraints

The project must avoid:

- Broken or private GitHub repository
- Deployment returning 404 or 500 errors

- Empty/mock repository
 - Plagiarized or duplicated codebase
 - Banned prebuilt UI or animation libraries
 - Hardcoded pricing values in the UI
 - Missing source code for core features
-

16. Recommended Implementation Checklist

- ☐ Set up project using an allowed framework
- ☐ Configure fonts from provided font list
- ☐ Configure color palette as CSS variables or design tokens
- ☐ Build semantic page shell
- ☐ Implement hero section
- ☐ Integrate provided SVG/UI assets
- ☐ Build Bento-grid desktop feature section
- ☐ Build mobile accordion version
- ☐ Implement active index persistence across breakpoints
- ☐ Build pricing matrix object
- ☐ Implement monthly/annual toggle
- ☐ Implement INR/USD/EUR currency switcher
- ☐ Ensure localized price-only updates
- ☐ Add social proof section
- ☐ Add footer
- ☐ Add SEO and OG metadata
- ☐ Validate responsive breakpoints
- ☐ Audit dependencies for banned libraries
- ☐ Test performance in Chrome DevTools
- ☐ Deploy live site
- ☐ Record demo video under 100MB
- ☐ Submit GitHub, deployment, and demo links